Freshers Final Capstone: Cloud Native DevOps Pipeline

Final Capstone Evaluation

Audience: Freshers completing the common tracks
Track: AWS EKS or Azure AKS (based on enrollment)
Deliverable: Production ready microservices infrastructure with CI/CD automation and cloud integration

1. Project Overview

Your goal is to demonstrate a **complete DevOps lifecycle** from infrastructure provisioning to automated deployment. You will build and deploy a microservices application on Kubernetes with Infrastructure as Code, GitOps CI/CD practices, and cloud native integrations.

Application: Candidates choose their own backend technology and architecture. The application must consist of at least 2 independent services (e.g. an API service and a worker or data service). Language and framework are open — Node.js, Python, Go, Java, .NET, or any other stack is acceptable as long as the choice is justified.

Scope: The architecture must include:

- **Infrastructure:** VPC/Network, EKS/AKS cluster, at least one managed cloud service (database, storage, queue, etc.), provisioned via Terraform
- **Application:** At least 2 containerized microservices with multiple replicas
- **Automation:** GitHub Actions CI/CD with build, test, security scanning, and deployment stages
- **Cloud Integration:** At least one pod connecting to a managed cloud service using IAM roles / Workload Identity (no static credentials)

Optional but encouraged:

- Event driven communication between services (message queue, pub/sub, or stream processing)

- An AI/ML component — e.g. calling a language model API, running inference, or integrating a managed AI service

Candidates may use technologies outside the training stack, provided they meet core requirements and choices are justified.

2. Technical Stack Requirements

Component	Requirement	Options
Infrastructure as Code	Terraform	AWS / Azure templates
Containerization	Docker	Multistage builds mandatory
Orchestration	Kubernetes	EKS (AWS) or AKS (Azure)
CI/CD	GitHub Actions	Build, Test, Deploy pipelines
Security	Scanning & Gates	SAST, Container scanning, Code quality
Application	Microservices	Any backend language/framework
Data / Messaging	Candidate's choice	Preferably a managed cloud service — database, object storage, queue, cache, etc.
Storage	S3 (AWS) / Storage Account (Azure)	Recommended; any managed storage service acceptable
Registry	ECR (AWS) / ACR (Azure)	Container image storage

3. Core Requirements

A. Infrastructure as Code (Terraform) - 25%

Minimum Requirements:

Track A: AWS

- [ ] VPC with public and private subnets (multi-AZ)
- [ ] EKS cluster (v1.29+) with managed node groups (2-3 nodes, auto scaling)
- [ ] At least one managed cloud service accessible from pods (e.g. RDS, DynamoDB, S3, SQS — candidate's choice)
- [ ] IAM roles for IRSA (IAM Roles for Service Accounts)
- [ ] CloudWatch log groups for monitoring
- [ ] ECR repository for container images
- [ ] Remote state backend (S3 with DynamoDB locking)

Track B: Azure

- [ ] Virtual Network with public and private subnets
- [ ] AKS cluster (v1.29+) with system & user node pools, auto scaling
- [ ] At least one managed cloud service accessible from pods (e.g. Azure Database, Storage Account, Service Bus, Cosmos DB — candidate's choice)
- [ ] Managed Identity for pod authentication
- [ ] Azure Container Registry (ACR) for images
- [ ] Application Insights for monitoring
- [ ] Remote state in Storage Account with locking

Code Quality Standards:

- [ ] Modular structure: `modules/vpc`, `modules/eks-or-aks`, `modules/database`, `modules/storage`
- [ ] `terraform validate` and `terraform fmt` pass with no errors
- [ ] No hardcoded values; use `terraform.tfvars` for configuration
- [ ] All resources tagged with `Environment` and `Owner`
- [ ] `.gitignore` excludes `*.tfstate` and sensitive files
- [ ] Outputs document cluster endpoints and credential information

B. Kubernetes Deployment & Configuration - 25%

Minimum Requirements:

- [ ] **Namespace Isolation:** Separate `production` namespace (not `default`)
- [ ] **Deployments:** At least 2 microservices deployed with more than 1 replica each — candidates must demonstrate how load is distributed across replicas and how state is managed (e.g.

external session store, stateless design, or persistent volumes where needed)

- [ ] **Service:** LoadBalancer or Ingress for external access
- [ ] **ConfigMap:** Environment variables (non sensitive config)
- [ ] **Secrets:** Database credentials stored securely in K8s Secrets (not hard coded)
- [ ] **ServiceAccount:** Linked to cloud IAM role / Workload Identity
- [ ] **Health Probes:** Liveness and Readiness probes on `/healthz` and `/ready` endpoints
- [ ] **Resource Limits:** CPU requests ≥100m, limits ≤500m; Memory requests ≥128Mi, limits ≤512Mi

Manifest Organization:

```
kubernetes/
├── namespace.yaml
├── deployment.yaml
├── service.yaml
├── configmap.yaml
└── serviceaccount.yaml
```

Validation:

- [ ] `kubectl apply -f kubernetes/ --dry-run=client` passes
- [ ] Pods in Running state across all services: `kubectl get pods -n production`
- [ ] Service has EXTERNAL-IP assigned
- [ ] Health endpoints on each service respond with 200 OK

C. GitHub Actions CI/CD Pipeline - 25%

Three Required Workflows:

1. Build Pipeline (`.github/workflows/build.yml`)

- Trigger: Push to main / develop
- Lint & Test application code
- Build Docker image
- Scan image for vulnerabilities (Trivy / GitHub Advanced Security)
- Push to ECR/ACR
- Fail on High/Severe issues

2. Deploy Pipeline (`.github/workflows/deploy.yml`)

- Trigger: Successful build completion
- Get cluster credentials (EKS / AKS)
- Update image tag in manifests
- Apply Kubernetes manifests (kubectl apply)
- Verify rollout status (kubectl rollout status)
- Run smoke tests
- Notification on success/failure

3. Infrastructure Pipeline (`.github/workflows/terraform-apply.yml`)

- Trigger: Changes to terraform/ folder
- terraform validate & plan
- Display plan for review
- Require manual approval
- terraform apply on approval

Requirements:

- [ ] All 3 workflows have successful recent executions
- [ ] Build pipeline has image scanning with security gate
- [ ] Deploy requires approval or environment gate
- [ ] No credentials exposed in logs
- [ ] GitHub Secrets configured for cloud credentials

D. Cloud Integration & Pod to Resource Communication - 25%

The core requirement here is demonstrating that pods communicate with at least one managed cloud service using IAM based identity — no static credentials, no secrets containing access keys. The specific service is the candidate's choice (database, object storage, queue, cache, etc.).

Track A: AWS IRSA (IAM Roles for Service Accounts)

Setup:

1. Create IAM role with appropriate permissions for the chosen service (via Terraform)
2. Create OIDC provider for EKS cluster
3. Annotate ServiceAccount with role ARN

Verification (REQUIRED):

- Application logs or a dedicated endpoint showing successful reads/writes to the cloud service
- No access key credentials in environment variables, Secrets, or code — identity must come from the pod's ServiceAccount

Track B: Azure Workload Identity

Setup:

1. Create managed identity with appropriate permissions for the chosen service (via Terraform)
2. Create federated credential linking pod to identity
3. Annotate ServiceAccount with client-id

Verification (REQUIRED):

- Application logs or a dedicated endpoint showing successful reads/writes to the cloud service
- No access key credentials in environment variables, Secrets, or code — identity must come from the pod's ServiceAccount

4. Evaluation Criteria

Pillar	Weight	Success Criteria
Infrastructure as Code	25%	Terraform modules reusable, validated, state managed, all resources tagged
Kubernetes Deployment	25%	All services running, manifests clean, health probes configured, resources limited
CI/CD Pipelines	25%	All 3 workflows operational, security gates active, approval required
Cloud Integration	25%	Pod connects to at least one managed cloud service via IAM/Workload Identity, no static credentials

Scoring Rubric

Score	Meaning	Criteria
5	Excellent	All requirements met, cloud integration working, pipelines fully operational, security practices in place
4	Good	Core infrastructure and deployments working, minor gaps in configuration or security
3	Satisfactory	Basic functionality present, some configuration gaps, partial cloud integration
2	Needs Improvement	Several components incomplete or misconfigured, cloud integration partially working
1	Unsatisfactory	Missing core components, cloud integration not working, pipelines incomplete

PASS: Score of 3 or above

5. Evaluation Process

Pre-Evaluation Checklist

- ☐ Repository shared with evaluators
- ☐ All code pushed to main branch
- ☐ No credentials in git history
- ☐ Terraform code passes validation
- ☐ Kubernetes manifests validated
- ☐ GitHub Actions workflows successful
- ☐ Application running and accessible ahead of the call

Final Evaluation Meeting

1. **Infrastructure Walkthrough** — Terraform modules, state, resource tagging (10 min)
2. **Live Demo** — Running services, CI/CD pipelines, Kubernetes deployment (15 min)
3. **Cloud Integration** — Pod identity, managed service connectivity (10 min)
4. **Q&A** — Architecture decisions, trade-offs, how things fit together (5 min)

The focus is on DevOps practices — infrastructure, deployment, automation, and cloud integration. Application business logic is not evaluated.

Total Time: ~40 minutes per candidate

7. Security & Code Quality Requirements

Mandatory Security Checks

- [ ] **No hard coded credentials** in any file or git history
- [ ] **Container doesn't run as root** - Non privileged user defined
- [ ] **Multistage Docker builds** - Separate build and runtime stages
- [ ] **Container image scanning** - Vulnerability scan before deployment
- [ ] **Secrets in Kubernetes** - Use K8s Secrets (not ConfigMaps)
- [ ] **RBAC configured** - ServiceAccount has minimal permissions
- [ ] **.gitignore complete** - Excludes `*.tfstate`, `.env`, credentials, etc.

8. Bonus Opportunities (+5% each)

- [ ] **Event Driven Architecture:** Services communicate via a message queue or pub/sub (e.g. SQS, Service Bus, Kafka, Redis Streams)
- [ ] **AI Integration:** A service calls a language model API, runs ML inference, or uses a managed AI service in a meaningful way
- [ ] **Advanced Monitoring:** Prometheus + Grafana dashboard
- [ ] **GitOps Deployment:** ArgoCD for declarative deployments
- [ ] **Cost Estimation:** Cloud cost analysis in Terraform outputs
- [ ] **Multi Region:** Infrastructure across multiple zones/regions
- [ ] **Disaster Recovery:** Backup/restore procedures tested
- [ ] **API Documentation:** OpenAPI/Swagger docs
- [ ] **Network Policy:** Kubernetes network policies

9. Originality & Integrity

Every submission must represent the candidate's own implementation. **Plagiarism is strictly prohibited.**

While AI assistance is permitted for:

- Generating application code (boilerplate)
- Explaining concepts
- Debugging

Candidates must:

- ☐ Demonstrate understanding of all code
- ☐ Customize solutions for their project
- ☐ Explain architectural decisions
- ☐ Complete implementation independently

Violations: Inadequate understanding or plagiarized code = automatic failure.

10. Submission Requirements

Deliverable	Description
Repository	All code pushed to main branch, shared with evaluators before the call
Final Evaluation	Live demo during the assessor meeting — see Section 5